

Instrucciones

- Lea con detenimiento y desarrolle **individualmente** cada una de las actividades a realizar durante la experiencia.
- Cree un archivo con extensión **.cpp** con lo desarrollado. El nombre del archivo debe tener el siguiente formato: **TEL102_C2_Nombre_Apellido.cpp** (Ej. TEL102_C2_Patricio.Olivares.cpp), sin incluir tildes.
- Las evaluaciones de la asignatura deben ser realizadas de manera **presencial** según lo estipulado en las reglas de clases y evaluaciones de la asignatura.
- Enviar el archivo a través de la página de aula del ramo, sección “Control 2” hasta las 12:05:00 del día Martes 03/10/2023 hora local continental de Chile (UTC-3).
- Trate de utilizar herramientas conocidas o aprendidas en clases. **No copie literalmente de recursos online.**
- Sea riguroso con las instrucciones de desarrollo.
- ¡Éxito!

1. Registro de empleados

Debido a su notable desempeño, la universidad ha decidido contratar sus servicios para implementar un programa de gestión de empleados. Debes diseñar una estructura de datos que permita almacenar la información de los empleados, incluyendo su nombre, número de identificación, y horas trabajadas por día. Para ello, ud. creará un programa tomando como base el código presente en el archivo *control2.cpp*. Recuerde que puede utilizar las funciones `std::cin.get()` para capturar los textos (ejemplo de uso en el siguiente [enlace](#)) y `std::cin.ignore()` para ignorar saltos de línea.

control2.cpp

```
#include <iostream>
#include <cstring>

struct employee{
    int id; // Identificador de empleado
    char nombre[100]; // Nombre del empleado
    int n_dias; // Cantidad de registros de horas
    int *horas_trabajadas; // Horas trabajadas
};

// Agrega aquí tus funciones

int main() {
    // Agrega aquí tu código principal

    return 0;
}
```

1. (30pts) Cree la función `student createEmployee()`, la cual solicita la información de un empleado y sus horas trabajadas por la terminal y entrega un elemento de tipo `employee` como retorno. El arreglo de horas debe ser un arreglo almacenado en **memoria dinámica**. Puede utilizar las funciones `std::cin.getline()` y `std::cin.ignore()` si así lo estima. Ej. de salida:

Salida (consola)

```
Ingrese nombre del empleado: Primer empleado
Ingrese identificación del empleado: 19
Ingrese la cantidad de días trabajados: 3
Ingrese Horas trabajadas el día 1: 8
Ingrese Horas trabajadas el día 2: 6
Ingrese Horas trabajadas el día 3: 4
```

2. (20pts) Cree la función `void updateHours(employee &empleado, int dia, int new_hours)` la cual recibe como entrada un empleado, el día que será actualizada y una cantidad de horas trabajadas, las que reemplazarán aquellas que se encuentran asignados para el día. Al utilizar el día, verifique que esta corresponda a una posición válida, considerando los días parten en 1.
3. (20pts) Cree la función `void showEmployeesInfo(employee *empleados, int n_empleados)`, la cual recibe un arreglo de empleados, así como la cantidad de empleados presentes en ese arreglo y muestra toda la información de dichos empleados. Ej. de salida:

Salida (consola)

```
Información de empleados
Empleado 1
Nombre del empleado: Primer empleado
Identificador del empleado: 19
Cantidad de días trabajados: 3
Horas trabajadas por día: 8 6 4
Empleado 2
Nombre del empleado: Segundo empleado
Identificador del empleado: 32
Cantidad de días trabajados: 2
Horas trabajadas por día: 4 6
Empleado 3
Nombre del empleado: Tercer empleado
Identificador del empleado: 41
Cantidad de días trabajados: 2
Horas trabajadas por día: 10 9
```

4. (30pts) Complete el código del main con las siguientes funcionalidades:

- Cree un arreglo de 3 empleados **en memoria dinámica**.
- Utilice la función `createEmployee()` para rellenar la información de cada empleado del arreglo.
- Solicite un identificador de empleado, una día dentro del arreglo de horas trabajadas para dicho empleado y una cantidad de horas trabajadas para dicha hora. Actualice el valor de horas trabajadas almacenadas en dicho día utilizando la función `updateHours()`. Suponga todos los valores ingresados como válidos (no necesita hacer verificaciones adicionales).
- Utilice la función `showEmployeesInfo()` para mostrar por la terminal la información de los empleados previamente llenados en el arreglo.

Ej. de salida:

Salida (consola)

```
Ingrese nombre del empleado: Primer empleado
Ingrese identificación del empleado: 19
Ingrese la cantidad de días trabajados: 3
Ingrese Horas trabajadas el día 1: 8
Ingrese Horas trabajadas el día 2: 6
Ingrese Horas trabajadas el día 3: 4
Ingrese nombre del empleado: Segundo empleado
Ingrese identificación del empleado: 32
Ingrese la cantidad de días trabajados: 2
Ingrese Horas trabajadas el día 1: 6
Ingrese Horas trabajadas el día 2: 6
Ingrese nombre del empleado: Tercer empleado
Ingrese identificación del empleado: 41
Ingrese la cantidad de días trabajados: 2
Ingrese Horas trabajadas el día 1: 10
Ingrese Horas trabajadas el día 2: 9
Actualización de horas trabajadas
Ingrese id del empleado: 32
Ingrese día trabajado: 1
Ingrese nueva cantidad de horas trabajadas: 4
Información de empleados
Empleado 1
Nombre del empleado: Primer empleado
Identificador del empleado: 19
Cantidad de días trabajados: 3
Horas trabajadas por día: 8 6 4
Empleado 2
Nombre del empleado: Segundo empleado
Identificador del empleado: 32
Cantidad de días trabajados: 2
Horas trabajadas por día: 4 6
Empleado 3
Nombre del empleado: Tercer empleado
Identificador del empleado: 41
Cantidad de días trabajados: 2
Horas trabajadas por día: 10 9
```